

Sieci Neuronowe i Uczenie Głębokie

Dropout.

Funkcja aktywacji ReLU.

Regularyzacja normą wektora wag

W metodzie kosztu wag każda waga jest ograniczana w sposób niezależny od pozostałych wag.

Na przykład, dla kosztu L^2 mamy:

$$J_r(y, t) = J(y, t) + \frac{\lambda}{2} \sum_k \sum_{ij} w_{ij}^{(k)^2}$$

A więc gradient dla pojedynczej wagi zanika, gdy:

$$w_{ij} = -\frac{1}{\lambda} \frac{\partial J}{\partial w_{ij}^{(k)}}$$

Amplituda wagi nie jest więc wprost powiązana z amplitudami innych wag w sieci.

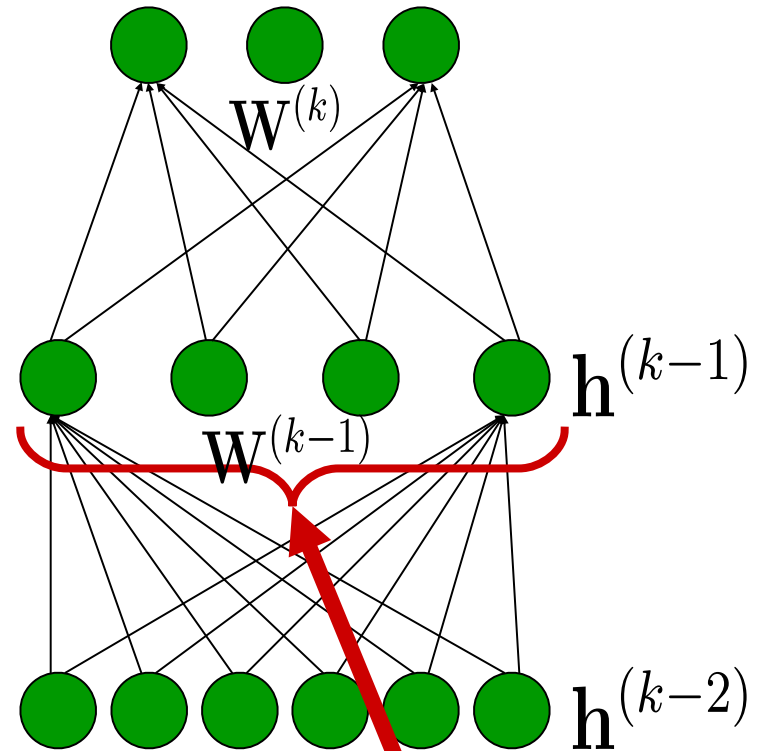
Regularyzacja normą wektora wag

Nie każda waga otrzymująca gradient funkcji kosztu ma pozytywny wpływ na efekt uczenia.

- Bo sieć może przeuczać.

Z poprzedniego wykładu pamiętamy jednak, że głównym celem kosztu wag jest ograniczenie całkowitego wejścia do kolejnej warstwy sieci.

- W ten sposób ograniczamy pojemność modelu.



$$z = h^{(k-2)} W^{(k-1)}, \quad h^{(k-1)} = f(z)$$

Regularyzacja normą wektora wag

Czy można zaproponować metodę regularyzacji, która:

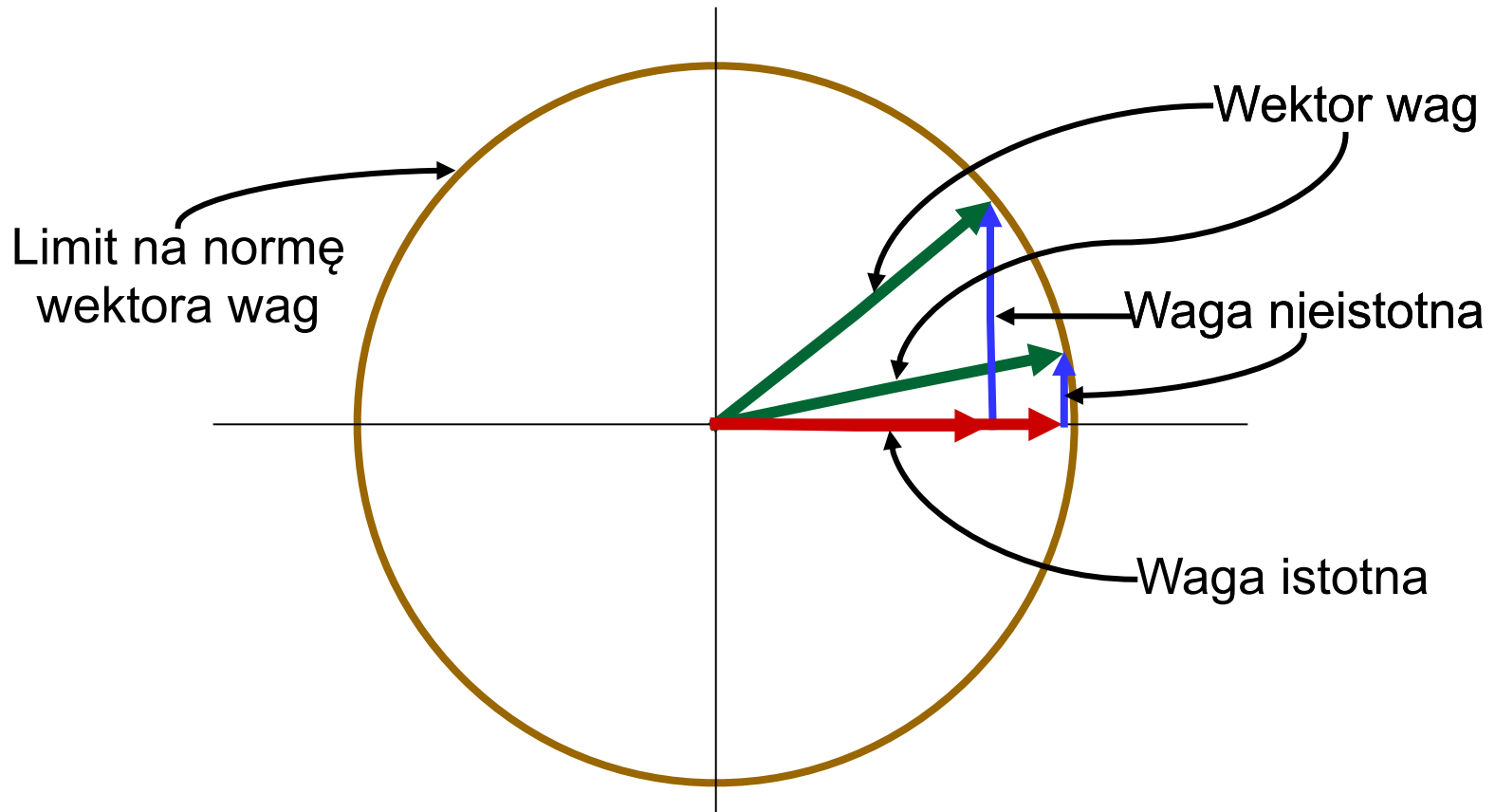
- będzie ograniczać całkowite wejście przekazywane (przez wagi) do kolejnej warstwy sieci,
- wymusi „konkurencję” pomiędzy wagami, tak by ważne wagi rosły kosztem wag nieistotnych?

Można: należy wprowadzić limit na normę wektora wag.

N. Srebro, A. Shraibman, In Proc. 18th annual conference on Learning Theory, COLT'05, pp. 545–560. Springer-Verlag, 2005.

Regularyzacja normą wektora wag

Dlaczego wprowadzenie limitu na normę wektora wag wymusza konkurencję pomiędzy wagami?



Regularyzacja normą wektora wag

Dodatkowym zyskiem z takiego sposobu regularyzacji jest poprawienie stabilności procesu uczenia.

- ❑ Żadna waga w sieci nie może mieć wartości bezwzględnej większej, niż przyjęty limit na normę wektora wag.
- ❑ Tak więc nawet przy dużym gradiencie uczącym (lub uczeniem z dużą wartością pędu) amplituda każdej wagi w sieci jest limitowana.
- ❑ Sieć nie powinna eksplodować numerycznie.
- ❑ Ma to istotne znaczenie zwłaszcza w początkowej fazie uczenia.

Regularyzacja normą wektora wag

Implementacja:

- Po każdym kroku uczenia znajdź wektory wag, których długość przekracza narzucony limit.
- Skróć te wektory w taki sposób, by ich długość była równa przyjętemu limitowi.

1. Wyznaczamy:

$$\rho_j = \begin{cases} 1 & \text{gdy } \|\mathbf{W}_{-,j}\| \leq \lambda \\ \frac{\lambda}{\|\mathbf{W}_{-,j}\|} & \text{gdy } \|\mathbf{W}_{-,j}\| > \lambda \end{cases}$$

gdzie:

λ – limit normy wektora wag

Regularyzacja normą wektora wag

Implementacja:

- ❑ Po każdym kroku uczenia znajdź wektory wag, których długość przekracza narzucony limit.
- ❑ Skróć te wektory w taki sposób, by ich długość była równa przyjętemu limitowi.

2. Poprawiamy macierz wag, tak by przywrócić limit na normę wektora wag:

$$W \leftarrow W \cdot \begin{bmatrix} \rho_1 & \rho_2 & \cdots & \rho_m \\ \rho_1 & \rho_2 & \cdots & \rho_m \\ \vdots & \vdots & \ddots & \vdots \\ \rho_1 & \rho_2 & \cdots & \rho_m \end{bmatrix}_{n \times m}$$

Dropout

IMPROVING NEURAL NETWORKS WITH DROPOUT

by

Nitish Srivastava

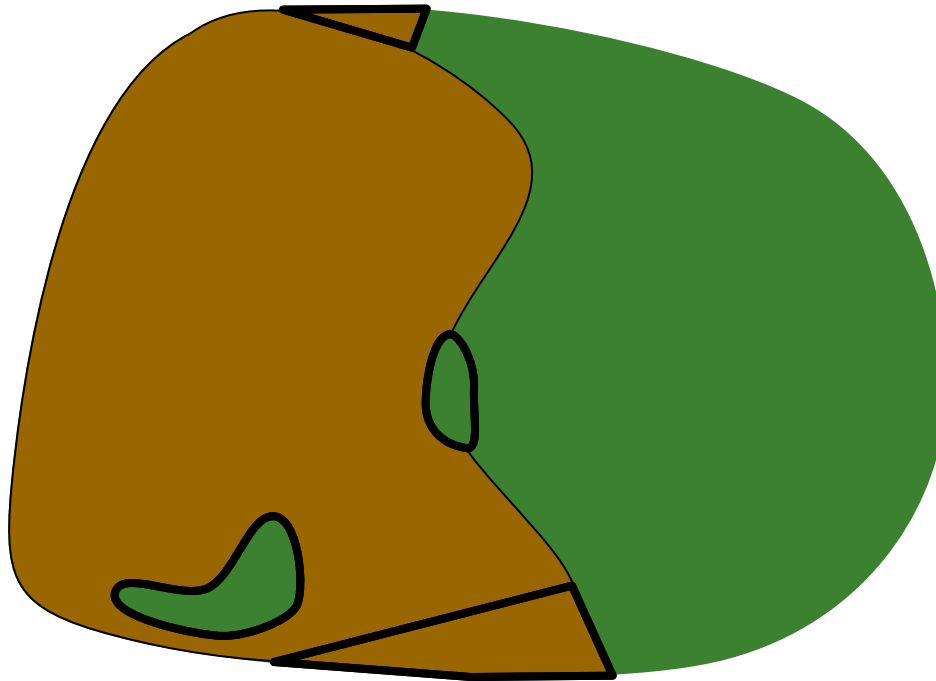
A thesis submitted in conformity with the requirements
for the degree of Master of Science
Graduate Department of Computer Science
University of Toronto

© Copyright 2013 by Nitish Srivastava

Dropout

Dobrym sposobem regularyzacji metod uczenia maszynowego jest uśrednianie odpowiedzi wielu modeli popełniających różne błędy.

- Na przykład przeuczających dla różnych obserwacji.



Dropout

Chcemy więc uczyć wiele modeli (rozwiązujących ten sam problem) w taki sposób, by modele te różniły się między sobą.

Jest to tzw. uczenie zespołowe (ang. *ensemble learning*).

Jak zapewnić, by wyuczone modele się między sobą różniły?

Jednym ze sposobów budowy modelu zespołowego jest **Bagging** (od *Bootstrap aggregating*).

Dropout

Bagging:

1. Ze zbioru trenującego wylosuj ze zwracaniem m przykładów uczących.
2. Potraktuj te przykłady jako zbiór trenujący i wyucz na nich (nowy) model.
3. Powtórz kroki 1 i 2 wielokrotnie.
4. Odpowiedzią modelu zespołowego będzie średnia z odpowiedzi wszystkich modeli wyuczonych w krokach 1-3.

Dropout można potraktować jako ekstremalną formę Baggingu.

Dropout

Dropout:

- ❑ Stwórz bardzo dużo sieci neuronowych.
- ❑ Każdej z tych sieci zaprezentuj jeden (losowo wybrany) przykład uczący.
- ❑ Ale jak sieć może się czegokolwiek nauczyć na jednym przykładzie uczącym?
- ❑ Sieci te będą współdzielić wszystkie parametry (wagi i biasy).

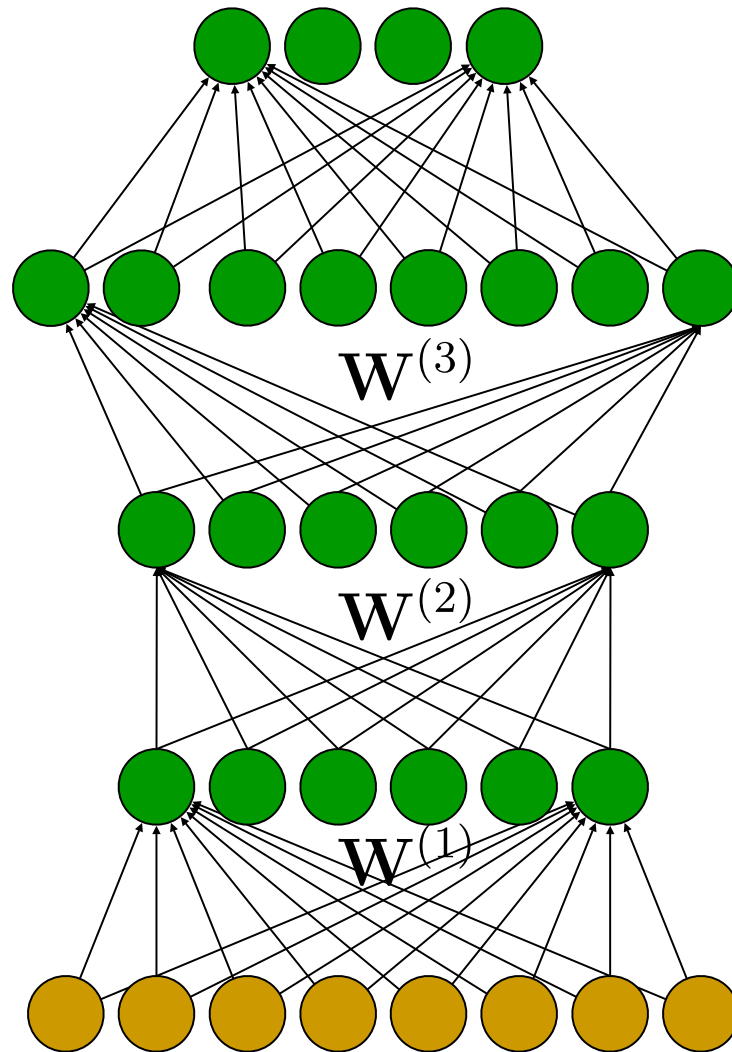
Jak w wydajny sposób zrealizować taki model zespołowy?

Dropout

Dropout - praktyczna realizacja:

- ❑ Tworzymy jedną sieć neuronową.
- ❑ Prezentujemy tej sieci kolejne przykłady uczące.
- ❑ Dla każdego przykładu uczącego, usuwamy z sieci losowo wybrane neurony w warstwie widocznej i warstwach ukrytych.
- ❑ Z reguły usuwamy 50% neuronów w każdej warstwie ukrytej.
- ❑ W warstwie widocznej usuwamy mniej neuronów, np. 20%.
- ❑ Oczywiście nie usuwamy neuronów w warstwie wyjściowej.

Dropout



*N. Srivastava, G.E. Hinton,
A. Krizhevsky, I. Sutskever,
R. Salakhutdinov, "Dropout: A
Simple Way to Prevent Neural
Networks from Overfitting",
Journal of the Machine Learning
Research 15,
pp. 1929–1958, 2014*

Dropout

Dropout – faza propagacji sygnału w algorytmie wstecznej propagacji błędów.

1. Dla każdej warstwy ukrytej wygeneruj losowy wektor maskujący (o długości równej liczbie neuronów w tej warstwie), w którym pewna część elementów (np. 50%) ma wartość 0 a reszta ma wartość 1:

$$\mathbf{D}^{(k)} = ([\text{rand}_1, \text{rand}_2, \dots, \text{rand}_{m_k}] > \eta_h)$$

gdzie:

η_h – odsetek neuronów usuwanych z warstw ukrytych

Dropout

Dropout – faza propagacji sygnału w algorytmie wstecznej propagacji błędów.

2. Podobny wektor wygeneruj dla warstwy widocznej. W tym wektorze odsetek zer powinien być mniejszy, niż w warstwach ukrytych (np. 20%):

$$\mathbf{D}^{(v)} = ([\text{rand}_1, \text{rand}_2, \dots, \text{rand}_{m_k}] > \eta_v)$$

gdzie:

η_v – odsetek neuronów usuwanych z warstwy widocznej

Dropout

Dropout – faza propagacji sygnału w algorytmie wstecznej propagacji błędów.

3. Propagując sygnał przemnóż wektory maskujące (element po elemencie) przez aktywacje neuronów i pochodne z aktywacji. Warstwę widoczną również przemnóż przez odpowiadający jej wektor maskujący.

$$h^{(1)} = f \left(\left(v \cdot D^{(v)} \right) W^{(1)} \right) \cdot D^{(1)}, \quad dh^{(1)} = f' \left(\left(v \cdot D^{(v)} \right) W^{(1)} \right) \cdot D^{(1)}$$

...

$$h^{(k-1)} = f \left(h^{(k-2)} W^{(k-1)} \right) \cdot D^{(k-1)}, \quad dh^{(k-1)} = f' \left(h^{(k-2)} W^{(k-1)} \right) \cdot D^{(k-1)}$$

$$y = g \left(h^{(k-1)} W^{(k)} \right), \quad dy = g' \left(h^{(k-1)} W^{(k)} \right)$$

Dropout

Dropout – wsteczna propagacja błędu.

4. Faza wstecznej propagacji błędu nie ulega zmianie. Przemnożenie aktywacji i ich pochodnych przez wektory maskujące jest równoważne usunięciu części neuronów z sieci.

$$\Delta^{(y)} = \nabla J_y(\mathbf{y}, t) \cdot d\mathbf{y},$$

$$\Delta^{(k-1)} = \Delta^{(y)} \mathbf{W}^{(k)T} \cdot d\mathbf{h}^{(k-1)},$$

$$\Delta^{(k-2)} = \Delta^{(k-1)} \mathbf{W}^{(k-1)T} \cdot d\mathbf{h}^{(k-2)},$$

...

$$\frac{\partial J}{\partial \mathbf{W}^{(k)}} = \mathbf{h}^{(k-1)T} \Delta^{(y)}$$

$$\frac{\partial J}{\partial \mathbf{W}^{(k-1)}} = \mathbf{h}^{(k-2)T} \Delta^{(k-1)}$$

$$\frac{\partial J}{\partial \mathbf{W}^{(k-2)}} = \mathbf{h}^{(k-3)T} \Delta^{(k-2)}$$

$$\frac{\partial J}{\partial \mathbf{W}^{(1)}} = \left(\mathbf{v} \cdot \mathbf{D}^{(v)} \right)^T \Delta^{(1)}$$

Dropout

W jaki sposób wykorzystamy taką sieć w przewidywaniu wartości docelowych?

- ❑ Moglibyśmy wielokrotnie propagować obserwację przez sieć, za każdym razem usuwając losowo wybrane neurony.
- ❑ Następnie uśrednilibyśmy uzyskane odpowiedzi sieci.
- ❑ Taki sposób przewidywania wartości docelowych byłby jednak bardzo wolny.
- ❑ Lepszym rozwiązaniem będzie przybliżenie przeciętnej odpowiedzi sieci z usuniętymi neuronami.

Dropout

W jaki sposób wykorzystamy taką sieć w przewidywaniu wartości docelowych?

- W tym celu można przemnożyć macierze wag przez odpowiadające im prawdopodobieństwa pozostania neuronu w sieci:

$$\mathbf{W}^{(k)} \leftarrow (1 - \eta_h) \mathbf{W}^{(k)},$$

$$\mathbf{W}^{(k-1)} \leftarrow (1 - \eta_h) \mathbf{W}^{(k-1)},$$

...

$$\mathbf{W}^{(1)} \leftarrow (1 - \eta_v) \mathbf{W}^{(1)}$$

- Odpowiedzi tak zmodyfikowanej sieci przybliżają przeciętną odpowiedź sieci z usuniętymi neuronami.

Dropout

Jakie efekty daje dropout?

- ❑ Dla MNIST sieć z dropoutem i limitem na normę wektora wag daje błąd na poziomie 0.95% – 1.06% (w zależności od rozmiaru sieci).
- ❑ **Ten wynik sieć osiąga bez pretreningu.**
Jeszcze mniejszy błąd można osiągnąć stosując dropout przy douczaniu sieci pretrenowanej.
- ❑ Dropout znacząco poprawia rezultaty również dla innych standardowych zbiorów testowych, obejmujących na przykład rozpoznawanie mowy czy klasyfikację dokumentów tekstowych.
- ❑ Obecnie dropout jest powszechnie stosowanym sposobem regularyzacji warstw w pełni połączonych (np. ostatnich warstw w niektórych sieciach konwolucyjnych).

Dropout

Uczenie sieci z dropoutem odbiega od typowego schematu uczenia sieci neuronowej:

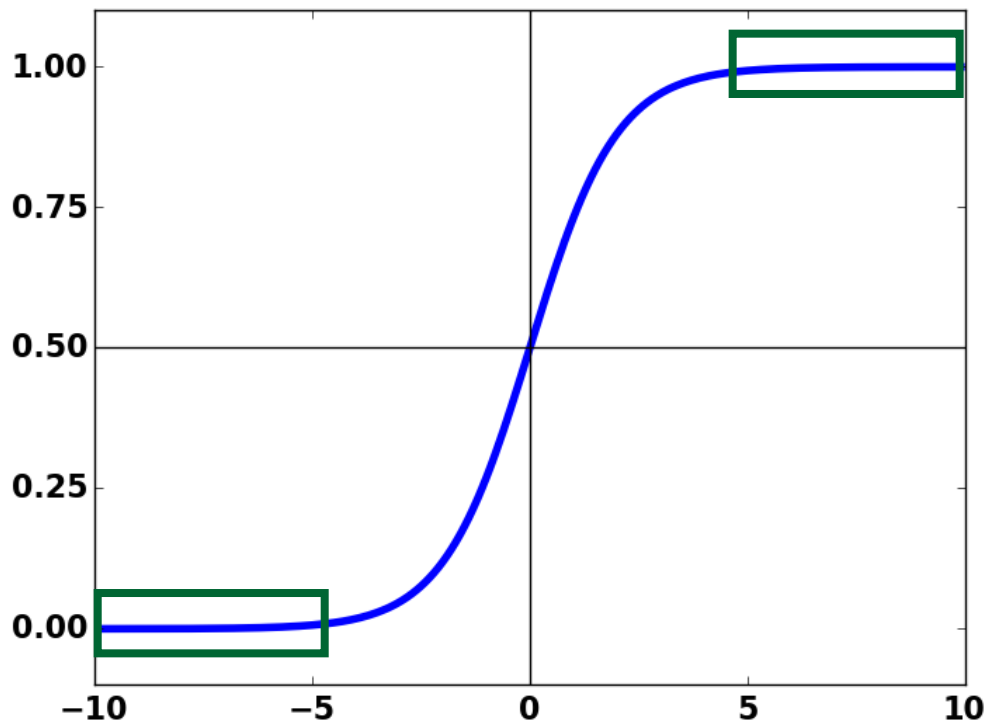
- ❑ Warto użyć znacznie większej sieci. Jeśli sieć nie przeucza na zbiorze trenującym to najpewniej lepsze wyniki uzyskamy ucząc większą sieć (która normalnie mogłaby przeuczyć) i stosując dropout.
- ❑ Dropout daje bardzo dobre rezultaty, gdy jest stosowany razem z regularyzacją za pomocą limitu na normę wag.
- ❑ W sieciach z dropoutem czasem używane są duże stałe uczące – nawet 10-100 razy większe niż w typowym uczeniu.
- ❑ Dodatkowo sieci te czasem uczone są z wysokim pędem – nawet do współczynnika pędu na poziomie 0.99.

Funkcja aktywacji ReLU

W sieciach omawianych do tej pory neurony w warstwach ukrytych wykorzystywały sigmoidalną funkcję aktywacji.

Ta funkcja aktywacji ma dwie wady:

- ❑ Neuron komunikuje (w przybliżeniu) jeden bit informacji: wartość 0 lub 1.
- ❑ Jeśli neuron jest nasycony (lub martwy), to pochodna sigmoidalnej funkcji aktywacji jest bliska zeru.



Funkcja aktywacji ReLU

Czy można zaprojektować funkcję aktywacji modelującą wartości: 0, 1, 2, 3, ...?

Można:

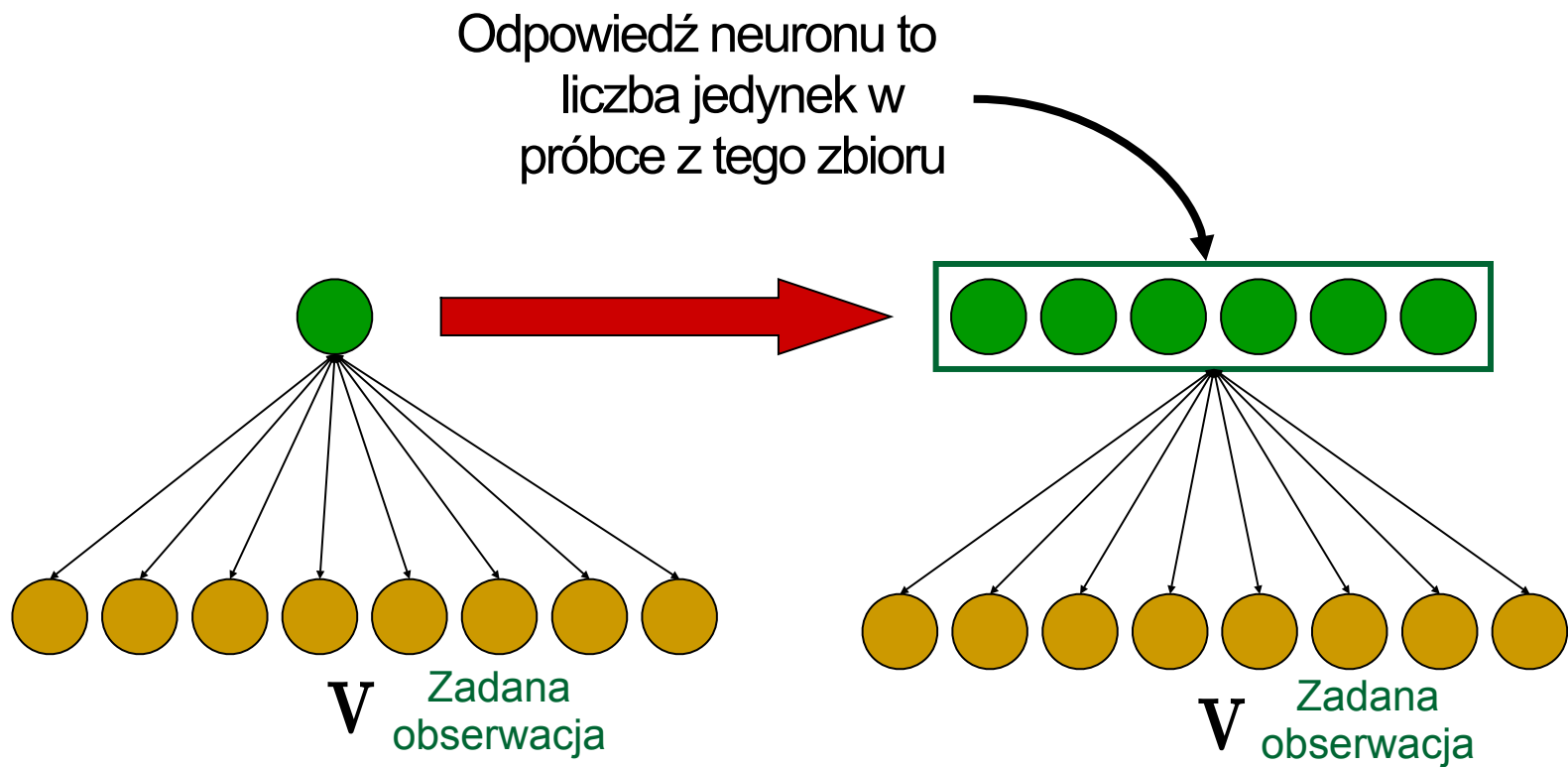
V. Nair, G.E. Hinton, "Rectified linear units improve restricted boltzmann machines", In: Proc. 27th International Conference on Machine Learning (2010)

Rozważmy RBM, w którym każdy neuron (sigmoidalny) w warstwie ukrytej zastąpiono zbiorem neuronów (również sigmoidalnych).

- ❑ Neurony w tym zbiorze mają taki sam wektor wag i bias.
- ❑ Jako wartość aktywacji takiego „kolektywnego” neuronu przyjmujemy liczbę jego elementów (a więc zwykłych neuronów sigmoidalnych), które w procesie próbkowania otrzymały wartość 1.

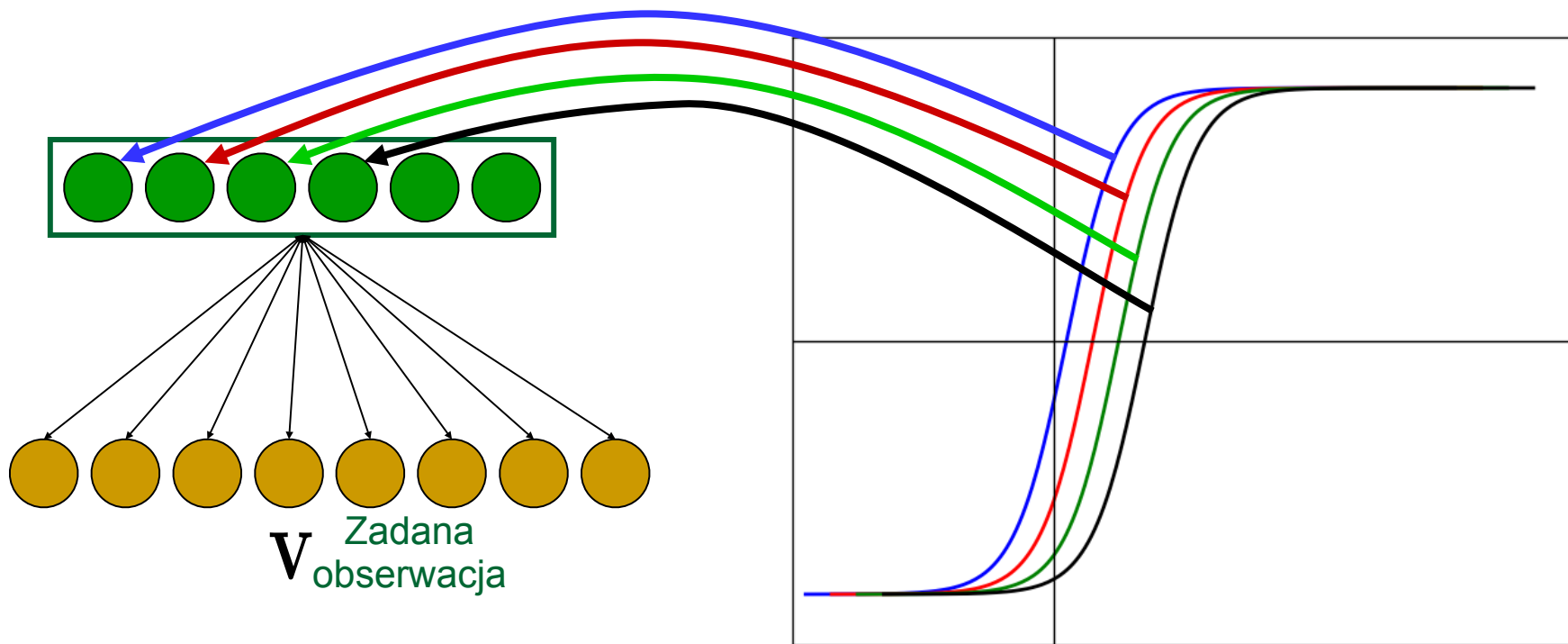
Funkcja aktywacji ReLU

Rozważmy RBM, w którym każdy neuron w warstwie ukrytej zastąpiono zbiorem neuronów.



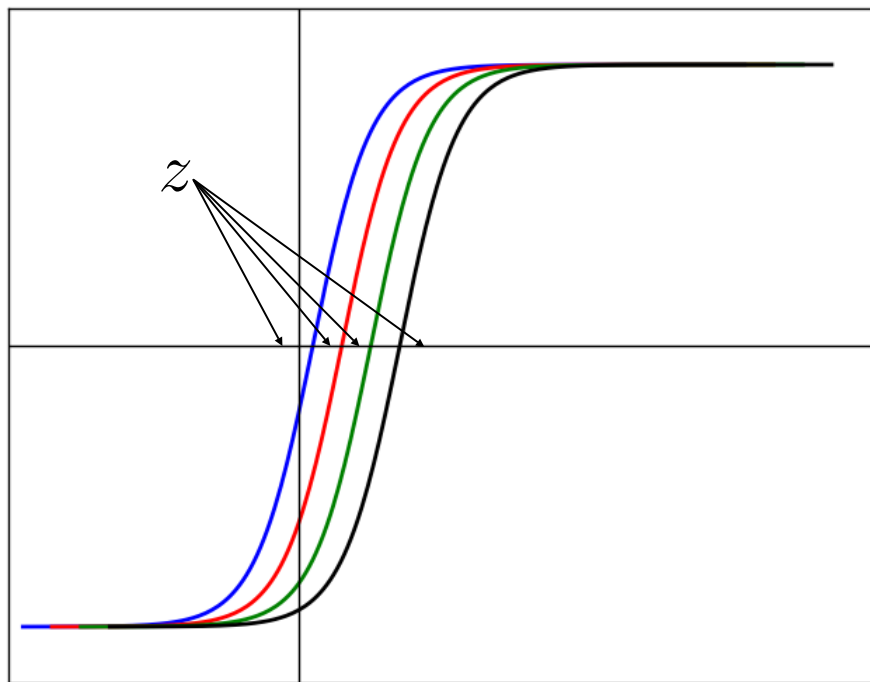
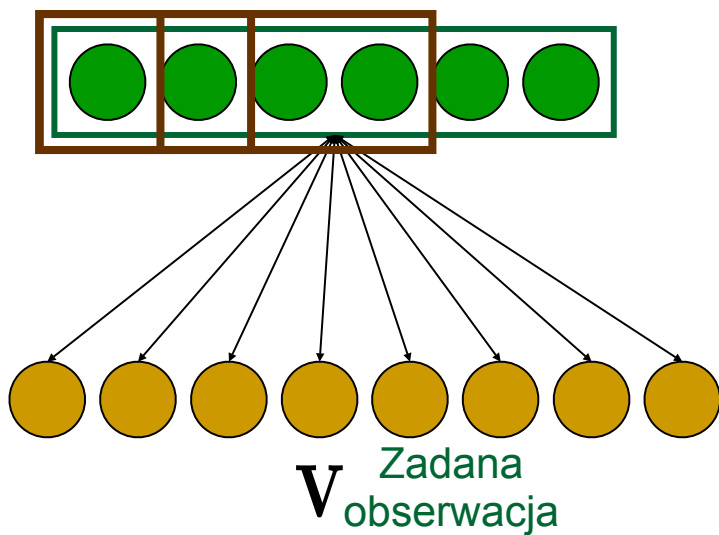
Funkcja aktywacji ReLU

Założmy teraz, że zmodyfikujemy bias każdego z neuronów w tym zbiorze w taki sposób, by liczba neuronów aktywnych rosła, gdy rośnie całkowite wejście do tworzonego przez nie neuronu „kolektywnego”.



Funkcja aktywacji ReLU

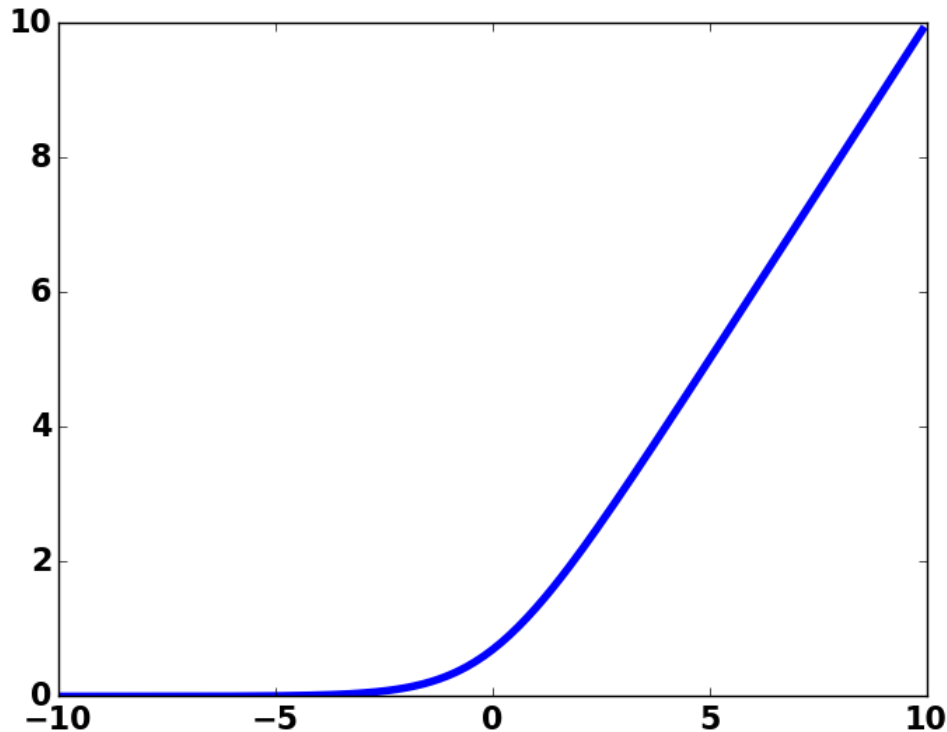
Założmy teraz, że zmodyfikujemy bias każdego z neuronów w tym zbiorze w taki sposób, by liczba neuronów aktywnych rosla, gdy rośnie całkowite wejście do tworzonego przez nie neuronu „kolektywnego”.



Funkcja aktywacji ReLU

Przeciętna wartość aktywacji neuronu „kolektywnego” będzie wówczas równa sumie wszystkich przesuniętych funkcji sigmoidalnych.

Dla dużej liczby elementów w neuronie „kolektywnym” otrzymamy:



Funkcja aktywacji ReLU

Ale jak wziąć próbkę z takiej funkcji aktywacji?

Sumę przesuniętych sigmoid przybliżamy w praktyce funkcją aktywacji ReLU (od ang. *Rectified Linear Unit*):

$$\text{ReLU}(z) = \max\{0, z\}$$

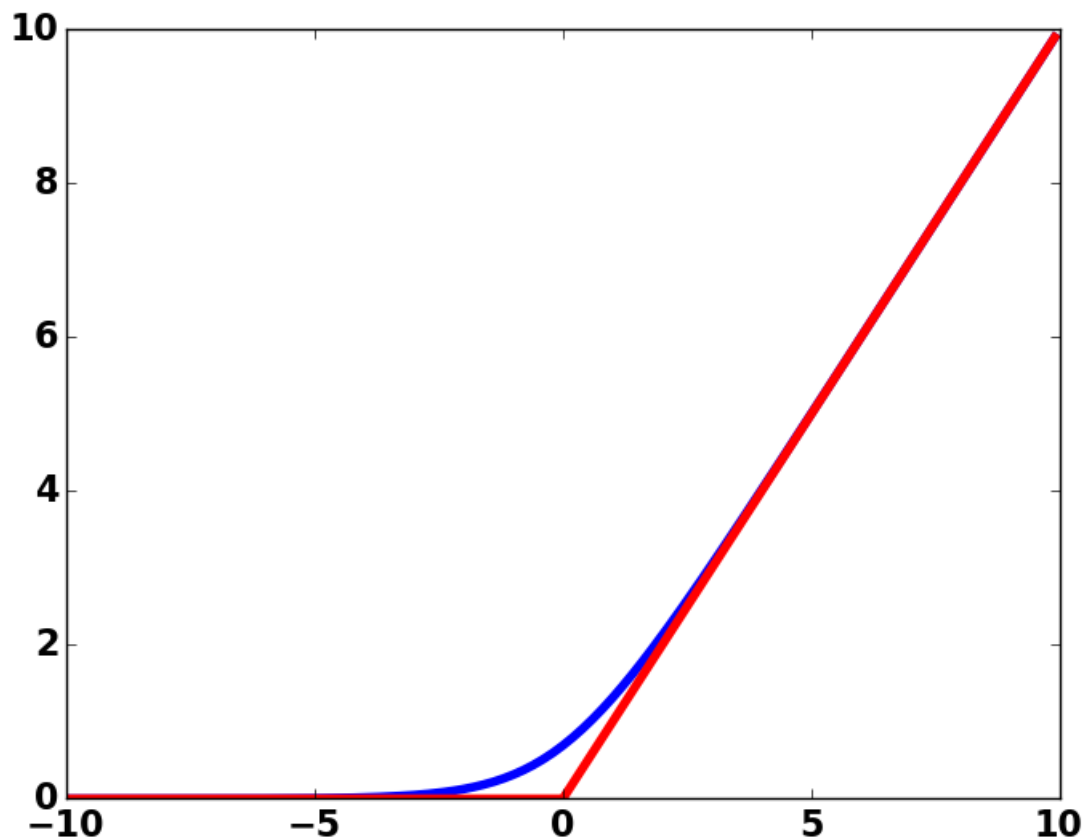
A próbkę z tej funkcji przybliżamy funkcję N-ReLU (od ang. *Noisy Rectified Linear Unit*):

$$\text{N-ReLU}(z) = \max\{0, z + N(0, \sigma(z))\}$$

gdzie $N(0, V)$ to liczba losowa z rozkładu Gaussa o wartości przeciętnej 0 i wariancji V .

Funkcja aktywacji ReLU

Sumę przesuniętych sigmoid przybliżamy w praktyce funkcją aktywacji ReLU



Funkcja aktywacji ReLU

Funkcje ReLU i N-ReLU można stosować jako aktywacje w warstwie ukrytej RBM.

- ❑ Algorytm CD-k nie ulega wówczas zmianie (z wyjątkiem zmiany funkcji aktywacji).

Funkcję ReLU można również stosować jako aktywację w warstwie ukrytej MLP.

- ❑ Algorytm wstecznej propagacji błędów także nie ulega zmianie.

Funkcja aktywacji ReLU

Zwróć uwagę, że:

$$z > 0 \Rightarrow \frac{d\text{ReLU}(z)}{dz} = 1$$

Gdy rośnie wejście do neuronu, ReLU nie ulega wysyceniu – gradient nie zanika.

Ma to istotne znaczenie w algorytmie wstecznej propagacji błędu:

$$\Delta^{(k-1)} = \Delta^{(k)} \mathbf{W}^{(k)T} \cdot f'(\mathbf{h}^{(k-1)})$$

Funkcja aktywacji ReLU

Obecnie ReLU jest jedną z najczęściej stosowanych funkcji aktywacji w sieciach MLP.

- ❑ Sieci z ReLU uczą się szybciej od sieci z sigmoidalną funkcją aktywacji.
- ❑ Często dają mniejszy błąd na zbiorze testowym.

Rezultaty wymienione w kontekście dropoutu uzyskano sieciami z funkcją aktywacji ReLU.

Kolejny wykład

Aspekty optymalizacyjne w uczeniu sieci neuronowych.
